

PROCESSOS ESTOCÁSTICOS II

Trabalho Cadeias de Markov

Apr 14 2026

Mateus Ribeiro

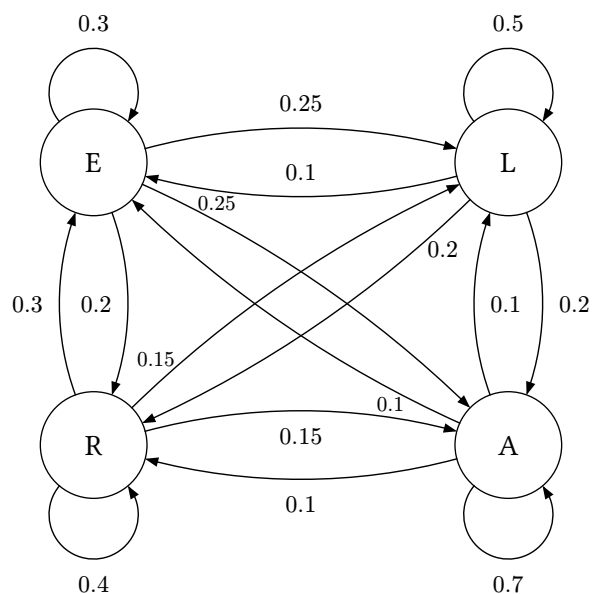
Prof. Charles Casimiro

- Obs: A resolução está dividida entre esse documento onde se encontram as questões teóricas e os resultados das computacionais e um jupyternotebook com os códigos necessários.

Questão 1

Mostre a representação gráfica dos estados da cadeia de Markov e suas transições possíveis

Dada a matriz de probabilidade de transição de um passo do sistema, temos que sua representação gráfica é dada por:



Questão 2

Faça um código, em qualquer linguagem da sua escolha, que faça as seguintes avaliações sobre o processo modelado pela cadeia de Markov descrito, assumindo a ordem dos estados como E,R,L,A.

- i. Mostre o valor da matriz de transição de n passos, $P(n)$, para $n = 1, 2, 5, 10, 20, 50, 100$.
 - ii. Mostre que a matriz de probabilidades de transição aponta para convergência ao longo das iterações.
 - iii. Para $n = 1, 2, 5, 10, 20, 50, 100$, calcular a probabilidade de estar em cada estado após n passos, começando de cada estado inicial arbitrário (de sua escolha).
 - iv. Qual estado tem maior probabilidade estacionária? Ilustre os valores do vetor de probabilidades estacionárias.
 - v. Se o sistema for iniciado com as seguintes probabilidades iniciais dos estados $p(0) = [0.3, 0.2, 0.1, 0.4]$ (novamente assumindo a ordem E, R, L, A), verifique quão rápido o sistema “esquece o estado inicial”? Ou seja, a partir de que momento o sistema aproxima das probabilidades estacionárias? Mostre os gráficos que lhe permitiram concluir esse ponto.
- i. Sabemos que, de acordo com a equação de Chapman-Kolmogorov, conseguimos obter a matriz de transição para um número genérico de $n + m$ passos através da seguinte relação:

$$p_{ij}(m+n) = \sum_k p_{ik}(m) \cdot p_{kj}(n)$$

De acordo com a equação acima, temos que:

$$\begin{aligned} P(n+m) &= \{p_{ij}(n+m)\} \\ \Rightarrow P(n+m) &= P(n) \cdot P(m) \\ \Rightarrow P(n) &= P^n = \prod_{i=1}^n P \end{aligned}$$

onde P é a matriz de transição de um passo.

Com o seguinte código em python, conseguimos realizar essas operações matriciais e obter os resultados.

```
# Definindo a matriz de transição de 1 passo
P = np.array([[0.3, 0.2, 0.25, 0.25],
              [0.3, 0.4, 0.15, 0.15],
              [0.1, 0.2, 0.5, 0.2],
              [0.1, 0.1, 0.1, 0.7]])

# Definindo uma lista para os valores desejados de "n"
ns = [1, 2, 5, 10, 20, 50, 100]
Ps = [np.linalg.matrix_power(P, n) for n in ns]
```

$$P^1 = \begin{pmatrix} 0.3 & 0.2 & 0.25 & 0.25 \\ 0.3 & 0.4 & 0.15 & 0.15 \\ 0.1 & 0.2 & 0.5 & 0.2 \\ 0.1 & 0.1 & 0.1 & 0.7 \end{pmatrix}$$

$$P^2 = \begin{pmatrix} 0.2 & 0.215 & 0.255 & 0.33 \\ 0.24 & 0.265 & 0.225 & 0.27 \\ 0.16 & 0.22 & 0.325 & 0.295 \\ 0.14 & 0.15 & 0.16 & 0.55 \end{pmatrix}$$

$$P^5 = \begin{pmatrix} 0.177 & 0.203 & 0.231 & 0.389 \\ 0.179 & 0.206 & 0.233 & 0.381 \\ 0.177 & 0.205 & 0.234 & 0.384 \\ 0.171 & 0.194 & 0.218 & 0.417 \end{pmatrix}$$

$$P^{10} = \begin{pmatrix} 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.398 \end{pmatrix}$$

$$P^{20} = \begin{pmatrix} 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \end{pmatrix}$$

$$P^{50} = \begin{pmatrix} 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \end{pmatrix}$$

$$P^{100} = \begin{pmatrix} 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \\ 0.175 & 0.2 & 0.227 & 0.397 \end{pmatrix}$$

- ii. Para demonstrar a convergência da matriz de transição ao longo das iterações, calcularemos a soma dos quadrados das diferenças entre os elementos das matrizes nos passos sucessivos da nossa amostra (por exemplo, a diferença entre a matriz no passo 100 e a do passo 50).

Ex.:

$$\sum_i \sum_j (p_{ij}(100) - p_{ij}(50))^2$$

Ao plotar esses valores para os passos sucessivos, podemos observar que a soma dos quadrados das diferenças converge para 0.

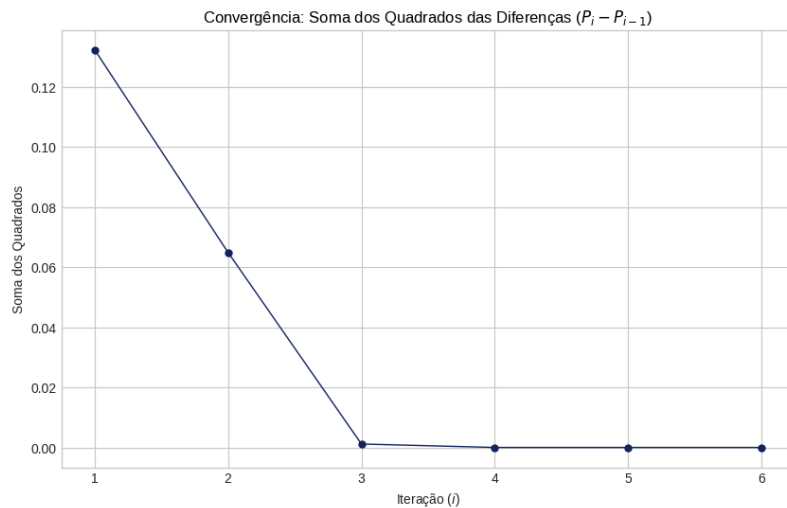


Figure 1: Convergence plot

Script em python utilizado para essa task:

```
from typing import List, Tuple

def calcular_diffs(Ps: List[np.ndarray]) -> Tuple[List[np.ndarray], List[float]]:
    # Lista para armazenar as matrizes de diferenca
    diffs = []
    for i in range(1, len(Ps)):
        diff = Ps[i] - Ps[i-1]
        diffs.append(diff)

    # Lista para armazenar a soma do quadrado dos elementos
    # das matrizes de diferenca
    diffs_unitaria = []
    for mat in diffs:
        aux = 0
        for i in range(len(mat)):
            for j in range(len(mat)):
                aux += (mat[i][j])**2
        diffs_unitaria.append(aux)
    return diffs, diffs_unitaria

diffs, udiffs = calcular_diffs(Ps)

# Definindo os xs para utilizar no plot
xs = np.arange(1, len(udiffs) + 1)
plt.figure(figsize=(10, 6))

plt.plot(xs, udiffs,
         marker='o',
         markersize=5,
         linestyle='-',
         linewidth=1,
         color="#152361",
         alpha=1.0)

plt.title(r'Convergência: Soma dos Quadrados das Diferenças ($P_i - P_{i-1}$)',
         color="#000000")

plt.xlabel('Iteração ($i$)')
plt.ylabel('Soma dos Quadrados')
```

iii.

- iv. Para calcular o vetor de probabilidades estacionárias temos que resolver o seguinte sistema de equações:

$$\pi \cdot P = \pi$$

Onde:

- π é o vetor de probabilidades estacionárias, ou seja, o vetor que queremos encontrar.
- P é a matriz de transição de um passo.

Ou seja,

$$\begin{pmatrix} \pi_E \\ \pi_R \\ \pi_L \\ \pi_A \end{pmatrix} \cdot \begin{pmatrix} 0.3 & 0.2 & 0.25 & 0.25 \\ 0.3 & 0.4 & 0.15 & 0.15 \\ 0.1 & 0.2 & 0.5 & 0.2 \\ 0.1 & 0.1 & 0.1 & 0.7 \end{pmatrix} = \begin{pmatrix} \pi_E \\ \pi_R \\ \pi_L \\ \pi_A \end{pmatrix}$$

Além disso, por se tratarem de probabilidades, sabemos que $\pi_E + \pi_R + \pi_L + \pi_A = 1$

v.